

Post-Test: Stored Functions Basic

Advanced Database - Pertemuan 3

Nama :Raihan Muhammad Riswandi

NIM:24110500001

Matkul :Database Advnaced

LAPORAN

Soal 1: Basic Arithmetic Function

Fungsi calculate_instructor_bonus(base_salary DECIMAL, performance_score INTEGER). Logic: menghitung bonus berdasarkan formula $\text{base_salary} * (\text{performance_score} / 100) * 0.15$. Jika input NULL, hasilnya 0.

```
CREATE OR REPLACE FUNCTION calculate_instructor_bonus(base_salary  
DECIMAL, performance_score INTEGER RETURNS DECIMAL AS $$
```

```
BEGIN
```

```
IF base_salary IS NULL OR performance_score IS NULL THEN
```

```
RETURN 0;
```

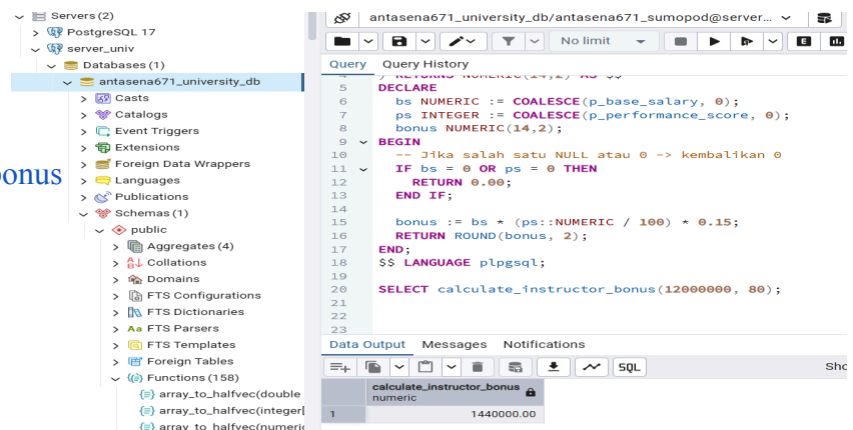
```
END IF;
```

```
RETURN base_salary * (performance_score / 100.0) * 0.15;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
SELECT calculate_instructor_bonus  
(12000000, 80);
```



Soal 2: Date Functions

Fungsi `calculate_service_years(hire_date DATE)`. Logic: menghitung lama pengabdian (tahun) dari `hire_date` hingga hari ini. Jika `hire_date` di masa depan, return 0.

```
CREATE OR REPLACE FUNCTION calculate_service_years(hire_date DATE)
```

```
RETURNS INTEGER AS $$
```

```
BEGIN
```

```
IF hire_date IS NULL OR hire_date > CURRENT_DATE THEN
```

```
RETURN 0;
```

```
END IF;
```

```
RETURN EXTRACT(YEAR FROM age(CURRENT_DATE, hire_date));
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
SELECT calculate_service_years('2010-08-15'); -- hasil: 15 (sampai 2025-09-19)
```

```
SELECT calculate_service_years('2026-01-01'); -- hasil: 0 (future date)
```

The screenshot shows a PostgreSQL database interface. On the left, the 'Databases' tree is expanded, showing the 'public' schema. The 'Functions' folder is selected, displaying a list of functions including `array_to_halfvec(double)`, `array_to_halfvec(integer)`, and `array_to_halfvec(numeric)`.

The main window displays the SQL query editor with the following code:

```
1 CREATE OR REPLACE FUNCTION calculate_service_years(p_hire_date DATE)
2 RETURNS INTEGER AS $$
3 BEGIN
4     IF p_hire_date IS NULL THEN
5         RETURN 0;
6     END IF;
7
8     IF p_hire_date > current_date THEN
9         RETURN 0;
10    END IF;
11
12    RETURN date_part('year', age(current_date, p_hire_date))::INT;
13 END;
14 $$ LANGUAGE plpgsql;
15
16 SELECT calculate_service_years('2010-08-15'); -- hasil: 15 (sampai 2025-09-19)
17 SELECT calculate_service_years('2026-01-01'); -- hasil: 0 (future date)
18
```

Below the query editor, the 'Data Output' tab is active, showing the results of the queries. The first query returns 15, and the second query returns 0.

calculate_service_years
integer
1
0

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.214'.

Soal 3: Inner Join dengan Aggregation

Fungsi `get_department_course_count(dept_id INTEGER)`. Logic: menghitung jumlah course dalam suatu department. Jika tidak ada department dengan ID tersebut, return 'Department not found'.

```
CREATE OR REPLACE FUNCTION get_department_course_count(dept_id INTEGER)
RETURNS TEXT AS $$

DECLARE

dept_name TEXT;

course_count INTEGER;

BEGIN

SELECT name INTO dept_name FROM departments WHERE id = dept_id;

IF dept_name IS NULL THEN

RETURN 'Department not found';

END IF;

SELECT COUNT(*) INTO course_count FROM courses WHERE department_id =
dept_id;

RETURN 'Department ' || dept_name || ': ' || course_count || ' courses';

END;

$$ LANGUAGE plpgsql;
```

The screenshot shows a database IDE interface. On the left, a tree view displays the database structure, including 'server_univ', 'Databases (1)', and 'antasena671_university_db'. The 'public' schema is selected. The main window displays the SQL code for the `get_department_course_count` function, which is identical to the code provided in the previous block. The 'Query' tab is active, showing the function definition. Below the code, the 'Data Output' tab shows the result of the function execution: 'Department [Computer Science]: 0 courses'.

Soal 4: Outer Join dengan Business Logic

Fungsi `get_instructor_workload(instructor_id INTEGER)`. Logic: menampilkan jumlah course yang diajar dan rata-rata tuition fee. Jika instructor tidak mengajar, tampilkan 0 courses, Avg Fee: 0.00. Jika instructor tidak ditemukan, return 'Instructor not found'.

```
CREATE OR REPLACE FUNCTION get_instructor_workload(instructor_id INTEGER)
```

```
RETURNS TEXT AS $$
```

```
DECLARE
```

```
instr_name TEXT;
```

```
course_count INTEGER;
```

```
avg_fee DECIMAL(10,2);
```

```
BEGIN
```

```
SELECT name INTO instr_name FROM instructors WHERE id = instructor_id;
```

```
IF instr_name IS NULL THEN
```

```
RETURN 'Instructor not found';
```

```
END IF;
```

```
SELECT COUNT(cs.id), COALESCE(AVG(cs.tuition_fee),0)
```

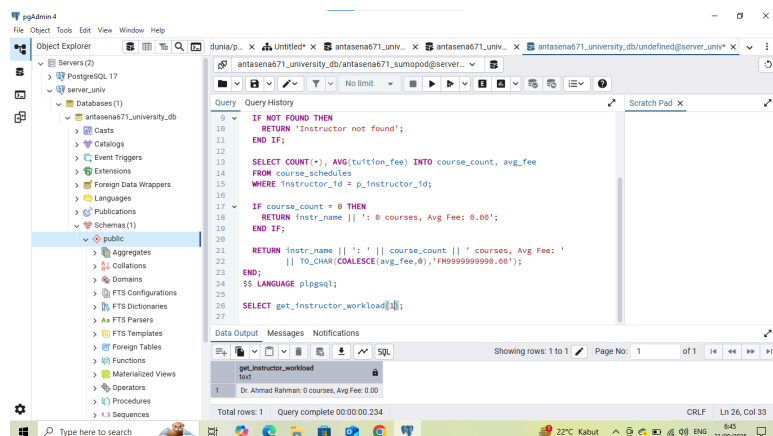
```
INTO course_count, avg_fee
```

```
FROM course_schedules cs
```

```
WHERE cs.instructor_id = instructor_id;
```

```
RETURN instr_name || ': ' || course_count || ' courses, Avg Fee: ' || TO_CHAR(avg_fee,  
'FM9999999999END;
```

```
$$ LANGUAGE plpgsql;
```



Soal 5: Complex Multi-Join Function

Fungsi `generate_department_summary(dept_code TEXT)`. Logic: membuat laporan ringkas department meliputi jumlah instructor, jumlah course, total tuition, dan rata-rata gaji instructor. Jika kode tidak valid → return 'ERROR: Invalid department code'.

```
CREATE OR REPLACE FUNCTION generate_department_summary(dept_code TEXT)
RETURNS TEXT AS $$

DECLARE

dept_name TEXT;

instr_count INTEGER;

course_count INTEGER;

total_tuition DECIMAL(12,2);

avg_salary DECIMAL(12,2);

BEGIN

SELECT name INTO dept_name FROM departments WHERE code = dept_code;

IF dept_name IS NULL THEN

RETURN 'ERROR: Invalid department code';

END IF;

SELECT COUNT(*) INTO instr_count FROM instructors WHERE department_id =
(SELECT id FROM departmen SELECT COUNT(*) INTO course_count FROM courses
WHERE department_id = (SELECT id FROM departments SELECT
COALESCE(SUM(cs.tuition_fee),0) INTO total_tuition

FROM course_schedules cs JOIN courses c ON cs.course_id = c.id

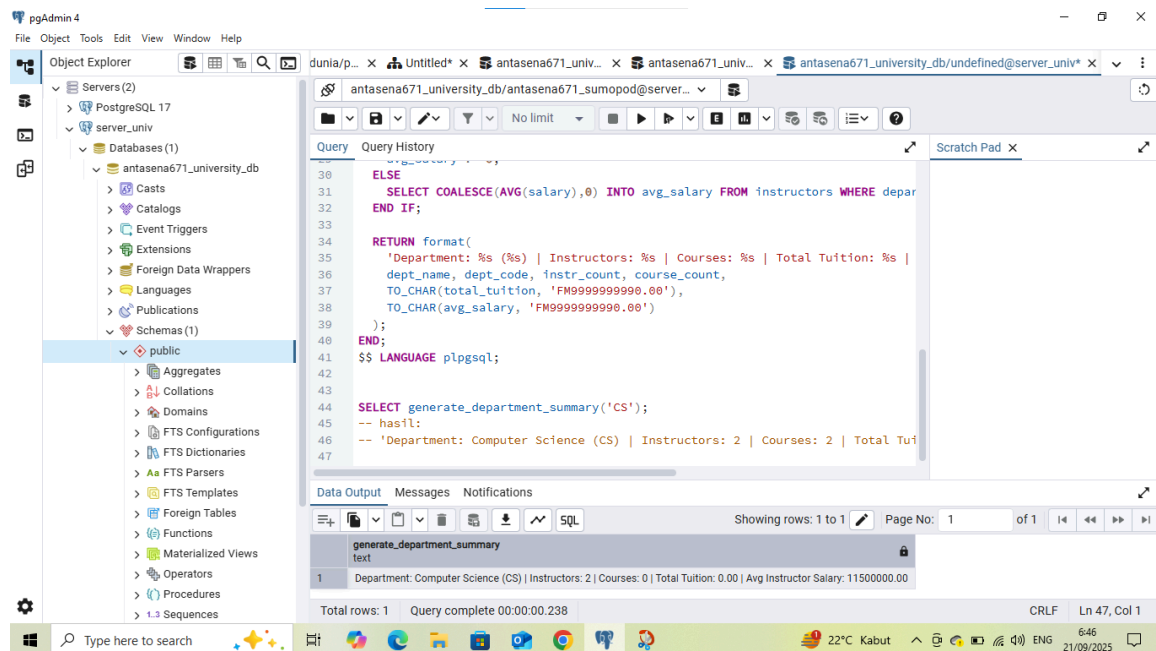
WHERE c.department_id = (SELECT id FROM departments WHERE code =
dept_code);

SELECT COALESCE(AVG(salary),0) INTO avg_salary FROM instructors WHERE
department_id = (SELECT id RETURN 'Department: ' || dept_name || ' (' || dept_code ||
') | Instructors: ' || instr_count ||
```

```
' | Courses: ' || course_count || ' | Total Tuition: ' || TO_CHAR(total_tuition, 'FM99999 '
| Avg Instructor Salary: ' || TO_CHAR(avg_salary, 'FM999999999.00');
```

END;

\$\$ LANGUAGE plpgsql;



Kesimpulan

Dari post-test ini, saya belajar bagaimana membuat stored function di PostgreSQL untuk berbagai kebutuhan: perhitungan aritmatika, manipulasi tanggal, penggunaan join, agregasi, hingga multi-join dengan aturan bisnis. Dengan latihan ini, kemampuan analisis data dan logika SQL saya meningkat, serta memahami bagaimana mengelola error handling dalam function.